

DeployBEV: A Self-Learned Matrix View Transformer for NPU-Deployable Multi-Fisheye BEV 3D Object Detection

Jiu Zou

Abstract—Surround-view fisheye cameras provide low-cost 360-degree perception for autonomous driving and automated parking, but state-of-the-art BEV 3D detectors are difficult to deploy efficiently on resource-constrained embedded NPUs. The view transformation stage in particular relies on operators such as `grid_sample`, two-dimensional gather, `scatterND`, one-hot construction, and data-dependent equality masks. These operators share a common characteristic: their memory access patterns are irregular and data-dependent, with random reads and writes scattered across feature maps. On embedded NPUs whose compute throughput comes from dense MAC arrays fed by DMA-driven blocked memory streams, such irregular access causes severe pipeline stalls, low MAC utilization, and poor cache behavior—the operators are not merely unsupported, they are fundamentally mismatched to the NPU’s dataflow architecture. This paper presents DeployBEV, a deployable self-learned BEV network that replaces explicit Lift-Splat-Shoot-style view transformation and BEV pooling with learnable separable matrix projections. The key insight is that image-to-BEV spatial correspondence can be learned as view-specific height and width projection matrices, rather than materialized as reference points, BEV mesh grids, or geometry-driven sampling indices. Matrix multiplication is the operator that NPUs are most efficient at: it exhibits regular blocked memory access, is compute-bound rather than memory-bound, and enables static tiling, pipelining, and INT8 quantization by the compiler. Unlike prior matrix-based BEV methods that derive fixed transport matrices from camera extrinsics, our projection matrices are fully learned from data and require no extrinsic parameters at inference time. The proposed view transformer applies the same matrix-projection principle to both image features and channel-wise-softmax-gated depth features, fuses them by element-wise multiplication, and aggregates multiple camera views by addition. Its core data path consists solely of matrix multiplication, permutation, softmax, element-wise multiplication, and addition—all operators with regular, statically analyzable memory access patterns. On the Fisheye3DOD CARLA benchmark with four surround-view fisheye cameras, DeployBEV achieves 0.4801 NDS, approaching the original Fisheye3DOD baseline of 0.4853 NDS with a gap of only 0.0052, while simultaneously reducing the total parameter count from 19.5M to 14.5M and replacing all memory-irregular operators in the view transformation path with NPU-native matrix operations.

Index Terms—Bird’s-eye-view perception, fisheye camera, 3D object detection, embedded NPU deployment, view transformation, memory access pattern.

I. Introduction

Surround-view camera 3D object detection estimates 3D bounding boxes from multiple vehicle-mounted images

and is a core perception task for autonomous driving, mobile robotics, and automated valet parking. Fisheye cameras are particularly attractive in this setting because four ultra-wide-angle lenses can cover the entire near-field region around a vehicle, where blind spots and tight maneuvering scenarios matter most. This paper studies the practical deployment setting in which four or more fisheye images are mapped to a bird’s-eye-view (BEV) representation and decoded into 3D boxes parameterized by $[x, y, z, w, l, h, \text{yaw}]$. In production automotive systems, accuracy alone is insufficient: a deployable model must also compile, quantize, and run efficiently on embedded NPUs that support a much narrower operator set than GPU training frameworks.

Modern camera-only 3D detectors commonly construct BEV features by projecting image features into a 3D or top-down space. Lift-Splat-Shoot (LSS) [1] lifts image features along discrete depth bins and splats them into a BEV grid, while BEVDet [2] develops a modular pipeline consisting of image encoder, view transformer, BEV encoder, and detection head. Fisheye3DOD [3] extends this line to surround-view fisheye cameras by introducing spherical modeling and releasing a CARLA-based synthetic benchmark. These methods establish BEV as a powerful unified representation, but the critical deployment bottleneck is the view transformation stage. The operations used—bilinear sampling via `grid_sample`, 2D tensor gather with data-dependent indices, `scatterND` for BEV pooling, one-hot depth construction, and equality-mask filtering—all share a structural property that is problematic for NPU deployment: their memory access patterns are irregular and input-dependent.

This irregularity is not a superficial compatibility issue. On a GPU, thousands of concurrent threads and large L2 caches can absorb scattered memory accesses with reasonable efficiency. An embedded NPU is architecturally different: its compute throughput comes from dense systolic MAC arrays that are fed by DMA engines streaming data in contiguous, tiled blocks through small on-chip SRAM. When a gather reads from arbitrary 2D locations determined by runtime-computed indices, or when `scatterND` writes to non-contiguous BEV cells, the DMA cannot predict or prefetch the next addresses, the MAC array stalls waiting for data, and the effective compute utilization drops to a small fraction of peak. Bilinear interpolation in `grid_sample` adds four scattered

Jiu Zou is with Goyu Meta Technology Co., Ltd., Wuhan, China (e-mail: jiuz@goyu-ai.com; 1069679911@qq.com).

reads per output pixel with data-dependent fractional coordinates. Even when these operators can be emulated on an NPU, they run as memory-bound scalar operations rather than as the dense vector-matrix operations the hardware is designed for, resulting in orders-of-magnitude lower throughput per watt. In our production implementation path targeting the Horizon J6 NPU, directly porting the public Fisheye3DOD-style network introduced multiple such operators including scatterND, two-dimensional gather, grid_sample, one-hot tensor generation, and data-dependent equality comparison masks; even replacing grid_sample with a two-dimensional gather formulation still violated the target NPU’s efficient operator subset. These practical obstacles motivated a different design question: can the view transformation be learned entirely by the network itself using only operations with regular, statically analyzable memory access—the kind of operations NPUs are actually built to accelerate?

We answer this question with DeployBEV, a self-learned matrix view transformer for deployable multi-fisheye BEV detection. Instead of generating lidar-to-camera or ego-to-camera extrinsic-based reference points, constructing a BEV mesh grid, and sampling image features via geometry-driven indices, our module learns view-specific projection matrices along the feature height and width dimensions. Matrix multiplication is the canonical NPU-friendly operator: it accesses memory in regular contiguous blocks, is compute-bound rather than memory-bound, and can be tiled and pipelined statically by the compiler to keep MAC arrays saturated. Separate projection matrices are learned for the image feature branch and a depth gating branch whose per-pixel channel distribution is normalized by a channel-wise softmax before projection; their projected BEV tensors are fused by element-wise multiplication and then summed across camera views. This redesign converts the most problematic part of the BEV pipeline into standard matrix multiplication and element-wise arithmetic—operations with regular memory access patterns that are universally supported and well-suited to both PTQ and QAT quantization workflows.

The implementation uses a lightweight EfficientNet-B0 image backbone, a compact BiFPN neck, and a center-based detection head following CenterPoint [4]. The core method is implemented in the ViewTransformerFisheye class through the `_spatial_transfom_learn` function, where three capacity variants are selected by the `learning_point_version` hyperparameter. Version 1 performs direct axis-wise projection, version 2 introduces one hidden matrix rank per axis, and version 3 adds another hidden matrix for higher representational capacity. All three variants share the same deployment-friendly operator principle: no BEVPool, no grid_sample, no two-dimensional gather, and no scatterND-style feature placement.

Contributions. The main contributions of this work are summarized as follows:

- We identify irregular memory access as the root deployment bottleneck of BEV view transformation on embedded NPUs, rather than operator availability

alone. Operators such as grid_sample, 2D gather, and scatterND are slow on NPUs because their data-dependent random reads/writes underutilize the dense MAC arrays and DMA pipelines; we formulate operator-constrained view transformation as a first-class architectural design problem.

- We propose a self-learned separable matrix view transformer that replaces all memory-irregular operations—reference point generation, BEV grid sampling, LSS-style BEV pooling, grid_sample, two-dimensional gather, and scatterND—with matrix multiplication, the most NPU-efficient operator class. Unlike prior geometry-derived matrix methods, the projection matrices are fully learned from data and require no camera extrinsic parameters at inference time.
- We provide three capacity variants controlled by `learning_point_version`, all implemented with only matrix multiplication, tensor permutation, channel-wise softmax, element-wise multiplication, and addition. Every operator in the core path has a regular, statically analyzable memory access pattern, enabling efficient NPU compilation and INT8 quantization.
- We conduct experiments on the Fisheye3DOD benchmark showing that DeployBEV approaches the original Fisheye3DOD NDS while using a strictly NPU-friendly operator set, with only 0.0052 NDS gap between our best model and the baseline, and with a smaller total parameter count (14.5M vs. 19.5M).

II. Related Work

Camera-only BEV 3D detection. Camera-only BEV detectors transform image-view features into an ego-centric top-down representation before 3D prediction. LSS [1] introduced a differentiable lift-and-splat formulation for arbitrary camera rigs by explicitly predicting categorical depth distributions. BEVDet [2] showed that a modular design with separate image encoder, view transformer, BEV encoder, and 3D detection head can produce strong multi-camera detection accuracy. Query-based methods such as PETR [5] encode camera geometry through 3D positional embeddings, while BEVFusion [6] demonstrates the utility of unified BEV spaces for multi-sensor perception. BEVFormer [7] employs deformable attention to interact with image features from BEV queries. StreamPETR [8] further explores object-centric temporal modeling for efficient multi-view 3D detection. The view transformation stages of these methods typically rely on explicit geometry, spatial sampling, tensor indexing, or BEV pooling operations that are problematic for NPU deployment.

Efficient and deployment-oriented BEV. Several recent works address the deployment gap in BEV perception. MatrixVT [9] replaces BEVPool with a sparse Feature Transporting Matrix and Ring & Ray decomposition, generating BEV features through matrix multiplications—an important step toward regular-memory-access view transformation. PersDet [10] avoids feature sampling

by detecting directly in perspective BEV space. Fast-BEV [11] proposes a lightweight deployment-friendly view transformation along with multi-scale encoders and strong data augmentation for on-vehicle chips. NVAutoNet [12] uses simple linear layers and BEV lookup tables for image-to-BEV transformation, achieving 53 FPS on NVIDIA DRIVE Orin. WidthFormer [13] compresses image feature dimensions and reports 1.5 ms latency on Horizon Journey-5. Our work shares the matrix-multiplication-only operator principle with these deployment-oriented methods, but differs in three aspects. First, MatrixVT’s projection matrices are geometry-derived and fixed from camera intrinsics and extrinsics, whereas our matrices are fully learned from data and require no extrinsic parameters at inference time. Second, prior matrix-based methods target pinhole camera configurations on nuScenes; our method is designed and evaluated specifically on surround-view fisheye cameras with severe radial distortion. Third, we introduce a dual-branch separable projection with three controllable capacity variants, all sharing the same NPU-friendly operator set with regular memory access patterns.

Fisheye perception and datasets. Fisheye optics provide ultra-wide fields of view but require camera models beyond the standard pinhole assumption [14]. Fisheye3DOD [3] studies surround-view fisheye 3D object detection and releases a CARLA-based benchmark with both pinhole and fisheye camera arrays. FPD [15] focuses on valet-parking fisheye BEV perception and proposes a distortion-insensitive multi-task framework. FishEye8K [16] targets fisheye road-object detection for traffic surveillance scenarios. SynWoodScape [17] provides synthetic surround-view fisheye data for multiple near-field autonomous driving tasks. FisheyeGaussianLift [18] further explores calibrated geometric unprojection and differentiable splatting for BEV feature lifting in fisheye cameras. Our work focuses specifically on an NPU-deployable BEV transformation for 3D detection, rather than on dataset construction or geometry-aware accuracy improvement alone.

Center-based detection and efficient backbones. Center-based detectors represent objects by center points and regress 3D geometry from local features [4], which inspires the compact BEV detection head used in this work. EfficientNet [19] and EfficientDet [20] demonstrate that lightweight backbones and feature-fusion networks can offer favorable accuracy-efficiency trade-offs. However, small parameter count alone is not sufficient for embedded deployment: operators with irregular memory access can become the throughput bottleneck even when the total model size is small, because they leave the NPU’s MAC array idle while waiting for scattered data. DAL [21] also emphasizes practical deployment and portability in 3D detection, although in a LiDAR-camera fusion setting. Our contribution is complementary: we redesign the camera-to-BEV transform itself at the architectural level to use only operators with regular, dense memory access patterns—the operations that NPUs can execute at full throughput.

III. Method

A. Overview

The proposed DeployBEV network maps multi-fisheye images to BEV features without any explicit BEV sampling or geometry-driven indexing. As illustrated in Fig. 1, each input fisheye image is processed by a shared EfficientNet-B0 image encoder and a lightweight feature neck. A depth gating branch is produced in parallel from the same image-view features by a 1×1 convolution followed by a channel-wise softmax, yielding a normalized per-pixel gating distribution. The self-learned matrix view transformer projects both feature tensors into BEV space using learned view-specific projection matrices along the height and width dimensions. The projected image BEV tensor and depth BEV tensor are fused by element-wise multiplication, and all camera views are aggregated by summation. Every operation in this view transformation—matrix multiplication, permutation, channel-wise softmax, element-wise multiply, and addition—has a regular, contiguous memory access pattern that the NPU compiler can tile and schedule for dense MAC execution. The final fused BEV feature map is processed by a BEV encoder and a center-based decoder to predict 3D bounding boxes and class scores.

B. Problem Formulation

Let $\{I_i\}_{i=1}^N$ denote the input images from N surround-view cameras. In the Fisheye3DOD setting, $N = 4$ corresponding to front, left, rear, and right fisheye views. The shared image encoder produces feature maps $F_i \in \mathbb{R}^{C \times H \times W}$ for each view, and the parallel depth-like branch produces gating features $D_i \in \mathbb{R}^{C \times H \times W}$ in the main configuration. The detection head consumes a unified BEV feature tensor $B \in \mathbb{R}^{C \times H_b \times W_b}$ and outputs per-class heatmap scores and 3D box regression parameters.

A conventional geometry-driven view transformer typically computes a deterministic mapping from BEV grid cells or 3D reference points to 2D image coordinates using calibrated camera intrinsics and extrinsics, then samples image features at those coordinates and aggregates them into the BEV grid. This explicit geometric mapping is attractive because it directly encodes known camera geometry, but it also introduces operations whose memory access patterns are fundamentally incompatible with NPU dataflow: reference points must be generated from calibration parameters, 3D-to-2D projections must be converted into sampling grids or tensor indices, and features must be gathered from scattered 2D locations or scattered into BEV memory layouts. Each of these steps involves data-dependent random memory access that defeats DMA prefetching and leaves MAC arrays idle.

Our formulation replaces this entire explicit sampling pipeline with learned matrices optimized directly by the downstream detection loss. The learned matrices are not required to exactly model the calibrated camera projection; instead, they learn the spatial feature correspondence that is most useful for the 3D detector under the hard

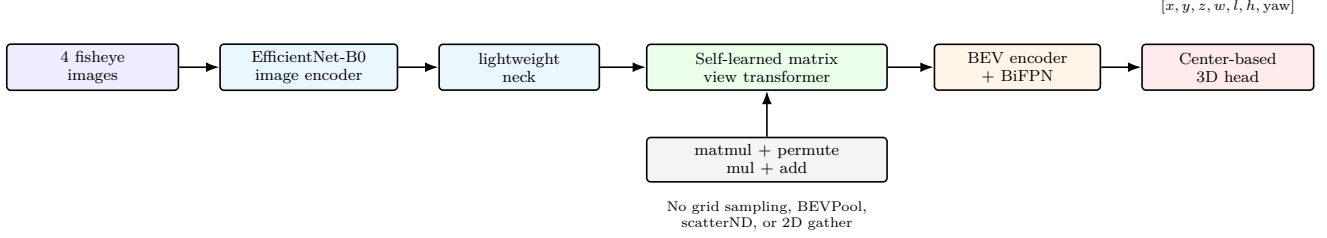


Fig. 1. Overall architecture of DeployBEV. The deployment-critical view transformation is implemented entirely by learned matrix projections and element-wise fusion, with no grid_sample, BEVPool, scatterND, or two-dimensional gather operators. Every operation in the shaded view-transform block has a regular, statically analyzable memory access pattern that maps to dense NPU MAC operations. Camera extrinsic parameters are not required during inference.

constraint of using only operators with regular memory access. This shifts the computational burden from runtime geometry calculations and dynamic indexing to trainable parameters that are fixed after training.

C. Self-Learned Separable Matrix Projection

For each camera view i , the proposed transformer learns separate width and height projection matrices. This axis-wise separable design is motivated by two key requirements. First, the transformation must be expressive enough to rearrange image-view evidence into a coherent BEV-like spatial layout. Second, it must avoid a full dense projection matrix whose parameter count and compute cost would scale as $O(HW \cdot H_b W_b)$, which would be prohibitive for practical feature map sizes.

In the default version 2 configuration (learning_point_version=2), the image feature projection path uses two-stage matrices along each spatial axis:

$$P_{i,1}^w \in \mathbb{R}^{W \times R}, \quad P_{i,2}^w \in \mathbb{R}^{R \times W_b}, \quad (1)$$

$$P_{i,1}^h \in \mathbb{R}^{H \times R}, \quad P_{i,2}^h \in \mathbb{R}^{R \times H_b}, \quad (2)$$

where R is the hidden matrix dimension controlling the projection capacity. The width dimension is first projected from W to W_b through the bottleneck, then a tensor permutation exposes the height dimension, which is similarly projected from H to H_b . This separable factorization reduces the complexity from $O(HWH_b W_b)$ to $O(WR + RW_b + HR + RH_b)$ per view, which is efficient.

For each channel c and spatial row h , the width dimension is projected by contracting the W axis with the width matrices:

$$\tilde{F}_{i,c,h,:} = F_{i,c,h,:} \cdot P_{i,1}^w \cdot P_{i,2}^w, \quad (3)$$

where $F_{i,c,h,:} \in \mathbb{R}^W$ is the row vector at channel c and height h , and the chain of matrix multiplications maps $W \rightarrow R \rightarrow W_b$. Collecting all channels and rows gives $\tilde{F}_i \in \mathbb{R}^{C \times H \times W_b}$. The height dimension is then projected similarly after permuting the tensor to expose the H axis:

$$(B_i^F)_{c,:,w_b} = \tilde{F}_{i,c,:,w_b} \cdot P_{i,1}^h \cdot P_{i,2}^h, \quad (4)$$

which maps $H \rightarrow R \rightarrow H_b$ for each channel c and projected column w_b , yielding the image BEV tensor $B_i^F \in \mathbb{R}^{C \times H_b \times W_b}$.

The depth gating branch mirrors the same separable matrix structure but uses independent learned matrices $Q_{i,1}^w$, $Q_{i,2}^w$, $Q_{i,1}^h$, and $Q_{i,2}^h$. Before projection, a 1×1 convolution produces a per-pixel multi-channel gating distribution $D_i \in \mathbb{R}^{C \times H \times W}$, and a channel-wise softmax normalizes it to a valid categorical distribution in the channel dimension. The normalized gating tensor is then projected through the same height/width matrix sequence to produce a depth BEV tensor B_i^D . Fusing the two branches by element-wise multiplication gives:

$$B_i = B_i^F \odot B_i^D, \quad (5)$$

where $\odot$ denotes element-wise multiplication. The per-view BEV tensors are aggregated across all cameras by summation:

$$B = \sum_{i=1}^N B_i. \quad (6)$$

This separable matrix formulation avoids constructing BEV grids, generating 3D reference points, applying camera extrinsic transformations, or indexing image features with two-dimensional gather operations. In the actual implementation, the entire forward data path consists of torch.matmul, permute, quantization stubs, channel-wise softmax, element-wise multiplication, and addition. The reason this operator set is efficient on NPUs is not merely that these operations are “supported”—it is that they all exhibit regular, statically analyzable memory access patterns. Matrix multiplication accesses operands in contiguous tiled blocks, allowing the DMA engine to prefetch data into on-chip SRAM in a predictable sequence and the MAC array to consume it at full throughput without stalling. Element-wise operations (multiply, add) are even simpler: they read and write tensors in linear order with unit stride, achieving near-peak memory bandwidth utilization. In contrast, grid_sample, 2D gather, and scatterND access memory at addresses determined by runtime-computed indices, which the DMA cannot predict, causing cache misses, pipeline bubbles, and severe MAC underutilization. By replacing the entire view transformation with matrix products and element-wise arithmetic, the module shifts from memory-bound random-access operations to compute-bound regular-access operations—the regime where NPUs deliver their rated throughput. We note that

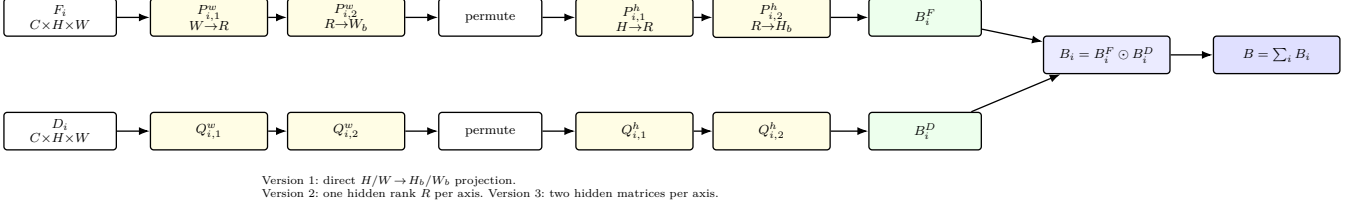


Fig. 2. Self-learned separable matrix view transformation. Version 2 projects width and height through a hidden rank R for both image features and depth-like gating features. Version 1 removes the hidden bottleneck (direct projection), and version 3 adds another hidden matrix per axis for increased capacity.

the design does not claim that camera geometry is unnecessary for perception; rather, it shifts the deployment-critical spatial mapping from runtime dynamic indexing operations into trainable parameters that are fixed after convergence.

D. Mathematical Foundation: The PAH Form

The original motivation of this work comes from the author’s initial observation in linear algebra: each channel of an image feature map is a matrix $A \in \mathbb{R}^{H \times W}$, each channel of the desired BEV feature map is also a matrix $Q \in \mathbb{R}^{H_b \times W_b}$, and the most direct algebraic device for transforming one matrix into another is simultaneous left and right multiplication,

$$Q = PAH, \quad (7)$$

where left multiplication by $P \in \mathbb{R}^{H_b \times H}$ acts as a row transformation along the height axis and right multiplication by $H \in \mathbb{R}^{W \times W_b}$ acts as a column transformation along the width axis. Concretely, row i of PA is $\sum_j p_{ij} a_j^\top$, a linear combination of the rows of A with coefficients from row i of P ; dually, column j of AH is a linear combination of the columns of A . The proposed separable matrix projection is precisely this PAH form with learnable P and H : in the implementation, the width-axis matmul realizes right multiplication (column transformation), and the height-axis matmul after the permutation realizes left multiplication (row transformation). From this perspective, the LSS-style lift-and-splat pipeline is only an indirect way of realizing a matrix-to-matrix mapping, and the view transformation needs nothing more than learned matrices.

Geometric reading of the implementation. Right multiplication displaces and accumulates features within each row independently: $(AH)_{ij} = \sum_k A_{ik} H_{kj}$, i.e., the feature at width position k of a row is moved with weight H_{kj} and accumulated into the new position j , with no cross-row mixing; left multiplication does the same within each column independently along the height axis, $(PA)_{ij} = \sum_k P_{ik} A_{kj}$. Jointly, $Q_{ij} = \sum_{k,l} P_{ik} A_{kl} H_{lj}$: every input location (k, l) contributes to every output location (i, j) with the separable weight $P_{ik} H_{lj}$, which is exactly why the 2D transport factorizes into a width pass and a height pass. Moreover, the learned matrices are keyed only by view (shape `[num_views, ...]`) and are broadcast over both the batch and channel dimensions in `torch.matmul`; hence all C feature channels of one image share the

same P and H . Spatial displacement is therefore channel-agnostic, while channel mixing is delegated to the 1×1 convolutions (`feat_net/depth_net`) before projection—a complementary division of labor: the convolutions mix channels without moving spatial positions, and the projection matrices move spatial positions without mixing channels.

Case 1: invertible P and H (elementary operations, matrix equivalence). The three types of elementary row operations correspond to invertible elementary matrices $E(i, j)$ (row swap), $E(i, c)$ (scaling), and $E(i, j(k))$ (adding a multiple of one row to another); a row operation is equivalent to left multiplication by such a matrix, and a column operation to right multiplication. By the equivalence canonical form theorem, any $A \in \mathbb{R}^{m \times n}$ of rank r admits invertible P, H such that

$$PAH = E_r = \begin{pmatrix} I_r & O \\ O & O \end{pmatrix}. \quad (8)$$

The proof is Gaussian elimination in matrix language: row operations reduce A to echelon form $P_s \cdots P_1 A = \begin{bmatrix} U_r \\ O \end{bmatrix}$; column operations use the pivots to clear the remaining entries, yielding $\text{diag}(D_r, O)$ with $D_r = \text{diag}(d_1, \dots, d_r)$, $d_i \neq 0$; left multiplication by $\text{diag}(d_1^{-1}, \dots, d_r^{-1}, 1, \dots, 1)$ scales the pivots to one. Accumulating all left factors into P and right factors into H preserves invertibility, since products of elementary matrices are invertible. It follows that $Q = PAH$ with invertible factors is reachable if and only if $\text{rank}(Q) = \text{rank}(A)$: necessity holds because invertible transformations are linear isomorphisms and preserve rank, and sufficiency holds because two rank- r matrices share the same canonical form, i.e., $A = P_A^{-1} E_r H_A^{-1}$ and $B = P_B^{-1} E_r H_B^{-1}$ imply $B = (P_B^{-1} P_A) A (H_A H_B^{-1})$. Hence the reachable set is $\{Q : \text{rank}(Q) = \text{rank}(A)\}$ —all same-shaped matrices of equal rank, not all matrices—and the invariant is the rank.

Case 2: arbitrary P and H . Since $\text{Col}(XY) \subseteq \text{Col}(X)$ and $\text{Row}(XY) \subseteq \text{Row}(Y)$, the lemma $\text{rank}(XY) \leq \min(\text{rank } X, \text{rank } Y)$ holds; applying it twice gives

$$\text{rank}(PAH) \leq \min(\text{rank } P, \text{rank } A, \text{rank } H) \leq \text{rank}(A). \quad (9)$$

Conversely, every Q with $\text{rank}(Q) = s \leq r$ is reachable: write $Q = P_Q E_s H_Q$, take the diagonal projection $D_s = \text{diag}(1, \dots, 1, 0, \dots, 0)$ with s ones, and use $E_s = D_s E_r D_s'$ together with $E_r = P_A A H_A$ to obtain

$Q = (P_Q D_s P_A) A (H_A D'_s H_Q)$. The reachable set is therefore $\{Q : \text{rank}(Q) \leq \text{rank}(A)\}$.

Geometric interpretation. A represents a linear map $T : V \rightarrow W$; right multiplication by an invertible H changes the basis of the domain V ($x = Hx'$), and left multiplication by an invertible P changes the basis of the codomain W ($y' = Py$). Since $\text{rank}(A) = \dim \text{Im}(T)$ is intrinsic to the map—independent of coordinate choices—a change of basis can never alter the dimension of the image, and no higher-rank map can be obtained through basis changes alone.

Implications for this design. With $H = W = 50$ and $H_b = W_b = 128$, the per-channel BEV feature satisfies $\text{rank}(Q) \leq 50 < 128$, an inherent ceiling of the pure PAH form. Three factors mitigate this ceiling in practice: the 64 channels carry complementary low-rank structures; the four-view summation increases rank because $\text{rank}(B_1 + B_2) \leq \text{rank}(B_1) + \text{rank}(B_2)$, so four $\text{rank} \leq 50$ matrices can jointly span the full 128-dimensional space; and the non-linearities of the downstream BEV encoder break the purely linear form. The same argument explains why the pure matmul chains of versions 2 and 3, lacking intermediate non-linearities, collapse mathematically into a single effective matrix (e.g., $P_2^\top P_1^\top$ is still 128×50) with the same rank ceiling as version 1; their accuracy gains stem from optimization dynamics rather than an enlarged reachable set, consistent with the ablation trend in Table IV.

Next steps (TODO). To turn A into an arbitrary matrix while preserving the regular-memory operator set, we plan two extensions with concrete validation protocols. (1) Affine form $Q = PAH + B$: a learnable offset matrix $B \in \mathbb{R}^{128 \times 128}$ can be full rank, so $\text{rank}(PAH + B)$ is no longer bounded by $\text{rank}(A)$ (in the extreme, $P = H = O$ and $B = I_{128}$ yield a full-rank Q); geometrically the linear map is upgraded to an affine one with translation freedom. The cost is only +16,384 parameters per view per branch ($\approx 0.13\text{M}$ for 4 views $\times$ 2 branches), addition is a regular-access operator, and the implementation reduces to adding `bias_bev` as a per-view nn.Parameter before the gating fusion. We will ablate the bias on versions 1–3 and visualize whether B learns a prior-occupancy structure. (2) Vector-space view: linear combinations $Q = \sum_{k=1}^K P_k A H_k$. By the identity $\text{vec}(AXB) = (B^\top \otimes A) \text{vec}(X)$ we have $\text{vec}(PAH) = (H^\top \otimes P) \text{vec}(A)$: the PAH form is only a Kronecker-structured linear map using 12,800 parameters to represent a map $\mathbb{R}^{2500} \rightarrow \mathbb{R}^{16384}$, whereas a general linear map would need $\approx 41\text{M}$ parameters. A sum of K terms corresponds to maps of Kronecker rank $\leq K$, which densely approximate arbitrary linear maps, and lifts the rank ceiling via $\text{rank}(Q) \leq \sum_k \text{rank}(P_k A H_k) \leq K \cdot \text{rank}(A) = 50K$ —already $K = 3$ permits full rank 128. Our current four-view summation and dual-branch fusion are implicit prototypes of this idea ($K=4$ across views); the extension refines it to within each view, with K implemented as an extra batch dimension via `bmm/einsum`. We will ablate $K \in \{1, 2, 3, 4\}$ to locate the NDS/mAP saturation point and test the combination

TABLE I
Matrix-projection capacity for the Fisheye3DOD configuration ($N = 4$, $H = W = 50$, $H_b = W_b = 128$, $R = 256$). Parameter counts include both image and depth-like projection branches across all four camera views.

Variant	Axis projection path	Matmul stages per branch	Projection params
Version 1	50 $\rightarrow$ 128	2	0.10M
Version 2	50 $\rightarrow$ 256 $\rightarrow$ 128	4	0.73M
Version 3	50 $\rightarrow$ 256 $\rightarrow$ 256 $\rightarrow$ 128	6	1.78M

$$Q = \sum_k P_k A H_k + B.$$

E. Capacity Variants

Version 1 uses a single learned projection matrix per spatial axis, directly mapping $H \rightarrow H_b$ and $W \rightarrow W_b$ without any hidden bottleneck. It has the smallest number of matrix multiplications and the lowest parameter count, serving as the minimal deployable baseline. Version 2 inserts one hidden dimension R per axis and is the default balanced training configuration. Version 3 adds an additional hidden matrix layer per axis ($H \rightarrow R \rightarrow R \rightarrow H_b$ and $W \rightarrow R \rightarrow R \rightarrow W_b$), increasing representational capacity while preserving exactly the same operator family with regular memory access. The purpose of these variants is to expose a controllable capacity-deployment trade-off: all three versions use only matrix multiplication and element-wise operations (all with regular, statically analyzable memory access), but they differ in the number of matrix multiply layers and trainable projection parameters.

Table I shows that the additional capacity of version 2 and version 3 is modest compared to the convolutional backbone and BEV encoder (which total roughly 10–20M parameters), but it directly increases the expressiveness of the learned view transformation. Version 2 is therefore a well-balanced default, while version 3 is useful when the learned spatial mapping requires higher capacity for more complex camera configurations.

F. Network Architecture and Training

The image backbone is EfficientNet-B0 [19] without the classification head, initialized with ImageNet pretrained weights. The image neck fuses the stride-16 (112-channel) and stride-32 (320-channel) backbone stages into 64-channel output tensors through a FastSCNN-style pyramid pooling and fusion module. The BEV encoder uses an EfficientNet-style lightweight backbone and a BiFPN neck [20] with 48 output channels. The decoder follows a center-based design [4] with heatmap classification and regression heads for center offset, height, 3D dimensions, orientation yaw, and velocity. Training uses Gaussian focal loss for center heatmaps and L1 loss for all box regression terms. This conventional detector design keeps the non-view-transform components standard, so the main experimental variable is the deployable construction of BEV features.

For the Fisheye3DOD experiments, the input image resolution is 800×800 , the image feature map size

TABLE II

Parameter count comparison between the original Fisheye3DOD and DeployBEV variants, measured directly from the trained checkpoint state dicts. For DeployBEV, the learnable view-transform (VT) matrices are the only additional parameters; the backbone, neck, BEV encoder, and detection head are shared.

Model	VT params (learnable)	Total params	Checkpoint size
Original Fisheye3DOD	0.05M [†]	19.48M	235.4 MB
DeployBEV v1	0.10M	12.79M	82.1 MB
DeployBEV v2	0.73M	13.41M	89.6 MB
DeployBEV v3	1.78M	14.46M	102.2 MB

[†]The original LSS view transform has only 0.05M learnable parameters but additionally materializes 1.44M non-learnable geometric buffers (frustum and sphere grids) derived from camera calibration. Checkpoint files include optimizer state for float training; the pure model weight size is roughly one half. Even the largest DeployBEV v3 remains 5.0M parameters (25.8%) smaller than the original model overall.

after the backbone is 50×50 ($16\times$ downsampling), and the BEV output grid is 128×128 . The physical BEV perception range is $51.2\text{m} \times 51.2\text{m}$ with 0.4m per cell resolution. The main configuration uses 64 feature channels, 64 depth channels, hidden projection dimension $R = 256$, AdamW optimizer with initial learning rate 2×10^{-4} , weight decay 0.01, and 36 float-training epochs with cyclic learning rate schedule. Training uses 4 GPUs with batch size 1 per GPU (effective batch size 4). The training and evaluation code is built around the Horizon OpenExplorer container environment (docker image `docker_open_explorer_ubuntu_22_j6_gpu_v3.8.1`) and J6 toolchain configuration; this paper reports float validation results since INT8 quantized board-side benchmarking on J6 is part of ongoing production work.

Table II compares parameter counts measured from the actual trained checkpoints. The entire trainable parameter overhead of the proposed matrix view transformer is only 0.10M to 1.78M parameters, depending on the capacity variant. Despite adding these projection matrices, all DeployBEV variants are smaller in total parameter count than the original Fisheye3DOD model (12.79–14.46M vs. 19.48M), because the original uses a heavier ResNet-18 image backbone with an FPN neck and a separate SECOND BEV backbone, and additionally stores 1.44M non-learnable geometry buffers (frustum and spherical grid tensors) that must be regenerated if the camera rig changes. The additional parameters of DeployBEV are concentrated entirely in the view transformation weight matrices, which are accessed as contiguous blocks during inference and are fixed after training.

IV. Experiments

A. Datasets and Evaluation Protocol

We evaluate DeployBEV on the Fisheye3DOD benchmark [3], which provides four surround-view fisheye camera images and 3D bounding box annotations in a CARLA simulation environment. Data preparation follows the public Fisheye3DOD workflow: first generate `fish-eye3dod_infos_train.pkl` and `fish-eye3dod_infos_val.pkl` using the official converter, then pack the images and annotations into LMDB format for efficient loading in the

TABLE III

Operator-level comparison of view transformation strategies. The key distinction is not merely operator availability but memory access pattern: irregular data-dependent access causes NPU pipeline stalls and low MAC utilization, while regular blocked access enables full-throughput execution.

Transform	Extrinsic needed	Irregular-memory ops	Memory access pattern	NPU throughput
LSS / BEVPool	Yes	scatter, gather, one-hot	Random read/write	Low
Grid sampling	Yes	grid_sample	4-pt scattered read	Low
Gather replacement	Yes	2D gather	Index-dependent read	Low
Proposed matrix	No	None	Regular blocked	High

The proposed DeployBEV view transformer uses only matrix

multiplication, permutation, channel-wise softmax, element-wise multiplication, and addition—all with regular, statically analyzable memory access that maps to dense MAC array operations.

TABLE IV

Fisheye3DOD validation results comparison. Higher is better for mAP and NDS; lower is better for error metrics. “Reg. mem.” indicates whether the view transform uses only regular-memory-access operators (matmul, element-wise) that run at full NPU throughput.

Method	Reg. mem.	mAP $\uparrow$	mATE $\downarrow$	mASE $\downarrow$	mAOE $\downarrow$	NDS $\uparrow$
Original Fisheye3DOD	No	0.3821	0.5906	0.1637	0.4799	0.4853
Ours version 1	Yes	0.3047	0.7160	0.1324	0.4489	0.4361
Ours version 2	Yes	0.3132	0.7202	0.1399	0.4448	0.4392
Ours version 3	Yes	<u>0.3533</u>	<u>0.6478</u>	0.1319	0.4209	0.4766
Ours version 3*	Yes	0.3515	0.6578	0.1333	0.3826	<u>0.4801</u>

Version 3 denotes a longer training run with improved orientation estimation. Underline marks the best value among DeployBEV variants.

Horizon training container. We use the standard nuScenes-style evaluation metrics [22]: mean Average Precision (mAP), mean Translation Error (mATE), mean Scale Error (mASE), mean Orientation Error (mAOE), and the aggregated nuScenes Detection Score (NDS). The codebase also includes a six-camera pinhole nuScenes path; preliminary results are reported in Section IV-F as a generality check.

B. Deployment-Oriented Design Analysis

Table III summarizes the deployment-relevant operator differences. The critical distinction is the memory access pattern. LSS/BEVPool, grid sampling, and gather-based transforms all rely on operations whose memory addresses depend on runtime-computed geometry (reference points, depth bins, or calibration-derived indices), producing scattered reads and writes that the NPU’s DMA cannot predict. The proposed matrix transform replaces all such operations with dense matrix products and element-wise arithmetic, whose memory access is regular, contiguous, and statically known at compile time. This is what enables the NPU to execute the view transformation at full MAC throughput rather than stalling on memory. The method also does not require any camera extrinsic parameters (lidar2camera, ego2camera) at inference time. The only trade-off is that the learned projection matrices are less geometrically interpretable than calibrated projection functions, but they are fully fixed after training and require no runtime geometry calculations.

C. Main Results on Fisheye3DOD

Table IV presents the main Fisheye3DOD validation results. Several observations can be made. First, our best

DeployBEV variant (version 3 with extended training) achieves 0.4801 NDS, which is only 0.0052 below the original Fisheye3DOD baseline of 0.4853, despite replacing all memory-irregular operators with NPU-native matrix multiplication. Second, all DeployBEV variants achieve better (lower) scale error (mASE) and orientation error (mAOE) than the original baseline, with version 3* reducing mAOE from 0.4799 to 0.3826. Third, the mAP gap is larger (0.3515 vs. 0.3821), indicating that the purely learned mapping has somewhat lower recall and precision for small or far objects, but the overall NDS is competitive because the other error metrics are better.

The NDS vs. mAP difference is important for proper interpretation. DeployBEV does not outperform the geometry-based baseline in raw detection AP, as reflected by the lower mAP. However, the NDS gap is minimal because localization, scale, and orientation errors remain competitive or superior. This supports a focused conclusion: the proposed learned matrix view transformer is a practical deployment-oriented alternative that achieves comparable detection quality while using only operators with regular memory access patterns—the operations that run at full throughput on NPU MAC arrays. It is not presented as a new accuracy state-of-the-art that claims to beat geometry-based methods on all metrics.

D. Class-wise Detection Results

The class-wise breakdown in Table V reveals that large rigid vehicles are detected more reliably than vulnerable road users. Bus and Cyclist achieve the highest AP scores (0.395 and 0.392), while Pedestrian remains the hardest class (0.252 AP) due to their small projected size in fisheye imagery and severe radial distortion at the image periphery. Cyclist detection is strong, likely because their distinctive shape and typical near-field locations make them easier to separate from background. This class imbalance is consistent with the original Fisheye3DOD results and is an inherent challenge of fisheye perception rather than a limitation specific to our view transformation design.

E. Ablation Study on Capacity Variants

The three capacity variants provide an ablation on the expressiveness of the learned view transformation. Version 1 (direct projection without bottleneck) achieves 0.4361 NDS, already demonstrating that even the simplest matrix projection can learn a reasonable BEV mapping. Version 2 with one hidden layer improves to 0.4392 NDS, a modest gain. Version 3 with two hidden layers brings a larger jump to 0.4766–0.4801 NDS, approaching the geometric baseline. This indicates that the view transformation benefits significantly from increased model capacity in the projection matrices, and that the two-hidden-layer version strikes a favorable balance between expressiveness and compute. Crucially, all three variants use the same NPU-efficient operator set with regular memory access; the additional capacity comes from more matrix multiply

layers, not from introducing any irregular-access operations. The projection parameters for version 3 total only 1.78M across all four views, which is negligible compared to the total model size.

F. Qualitative Results

Qualitative visualization on Fisheye3DOD validation samples shows that DeployBEV produces well-localized 3D boxes on surrounding vehicles in all four fisheye views. The BEV-level predictions exhibit consistent spatial arrangement across camera views, indicating that the learned matrices successfully capture the overlapping field-of-view relationships between adjacent cameras without explicit geometric supervision. The largest errors occur on distant pedestrians and partially occluded objects at the extreme fisheye periphery, which is consistent with the quantitative class-wise results.

G. Preliminary Generality Check on Pinhole nuScenes

To verify that the self-learned matrix view transformer is not specific to fisheye optics, we additionally trained the same architecture on the six-camera pinhole nuScenes benchmark [22] using the extended multi-layer matrix variants with intermediate 1×1 convolution–BN–ReLU blocks and residual connections between projection stages. Early-stage checkpoints reach 0.1717–0.1903 NDS and 0.0599–0.0788 mAP on the nuScenes validation split (Table VI). These numbers are far below state-of-the-art nuScenes detectors and are reported only as evidence that the same NPU-friendly operator set transfers across camera models and view counts without any architecture change; we do not claim competitive nuScenes accuracy, and longer training is ongoing.

V. Discussion

A. Why Simulation-to-Real Gap Exists

While DeployBEV achieves strong results on the Fisheye3DOD CARLA simulation, we observed that performance degrades on real-world data with multiple different extrinsic calibrations. This is expected and follows from the nature of our approach: because the spatial mapping is fully learned rather than derived from calibration parameters, the projection matrices overfit to the specific camera mounting geometry, distortion profile, and scene statistics present in the training data. When presented with real images that have different lens distortion characteristics, different exposure and color distributions, different camera extrinsics, or more complex real-world scene geometry, the fixed learned matrices cannot adapt the way a geometry-based method can by simply updating the calibration parameters. This is the primary trade-off of our deployment-first design: we gain NPU operator simplicity at the cost of reduced cross-calibration generalization.

This limitation can be addressed in practice by: (1) fine-tuning the projection matrices on real data for each specific vehicle platform, which is standard practice in

TABLE V
Class-wise Fisheye3DOD validation results for DeployBEV version 3* (best NDS checkpoint).

Class	AP $\uparrow$	ATE $\downarrow$	ASE $\downarrow$	AOE $\downarrow$	AP@0.5 $\uparrow$	AP@1.0 $\uparrow$	AP@2.0 $\uparrow$	AP@4.0 $\uparrow$
Car	0.3277	0.6691	0.2052	0.1491	0.0334	0.2043	0.4477	0.6252
Van	0.3547	0.5881	0.1938	0.1397	0.0595	0.2544	0.4848	0.6200
Truck	0.3876	0.6509	0.1150	0.2291	0.0433	0.2748	0.5416	0.6908
Bus	0.3952	0.6853	0.0032	0.2542	0.0322	0.2552	0.5407	0.7525
Pedestrian	0.2520	0.7736	0.0543	1.3926	0.0015	0.1401	0.3787	0.4876
Cyclist	<u>0.3919</u>	0.5794	0.2282	0.1313	0.0761	0.3099	0.5379	0.6436

TABLE VI
Preliminary nuScenes validation results (six pinhole cameras, extended matrix variants). Reported only as a generality check, not as an accuracy claim.

Variant	mAP $\uparrow$	NDS $\uparrow$
Extended v9	0.0599	0.1717
Extended v10	0.0788	0.1895
Extended v11	0.0773	0.1903

automotive production; (2) augmenting training with diverse camera mounting variations; or (3) adding a lightweight calibration-dependent adaptation layer on top of the learned base projection. These directions are left for future work.

B. Why Regular Memory Access Matters on NPUs

The deployment advantage of matrix multiplication over gather/scatter is not a matter of operator whitelists—it follows from the dataflow architecture of embedded NPUs. An NPU such as the Horizon J6 BPU achieves its TOPS rating through a systolic array of multiply-accumulate units that consume data in fixed-size tiles streamed from on-chip SRAM. The DMA engine that feeds this array works most efficiently when it can predict and prefetch contiguous blocks of memory. Matrix multiplication naturally satisfies this: both operands are accessed in sequential tile order, and the access pattern is known at compile time, enabling the compiler to perform tiling, double-buffering, and layer-fusion optimizations that keep the MAC array near full utilization.

In contrast, `grid_sample`, `2D gather`, and `scatterND` read or write at addresses computed from runtime data (e.g., projected reference point coordinates or depth bin indices). The DMA cannot prefetch these addresses, so each access may incur an off-chip DRAM round-trip. On a GPU this is tolerable because thousands of threads and a large L2 cache amortize the latency; on an NPU with a small scratchpad and in-order execution, the result is pipeline bubbles and MAC utilization that can drop below 10% of peak. This is why replacing these operators with matrix products is not merely a compatibility workaround—it is the difference between an operator that runs at the

NPU’s advertised throughput and one that bottlenecks on memory latency.

A secondary benefit is quantization. INT8 matrix multiplication is the most mature and heavily optimized operation in every NPU toolchain, with per-channel scale factors and hardware-supported rounding. Irregular-access operators, when they can be quantized at all, typically require per-element dequantization/requantization around each scalar load/store, which further degrades throughput and introduces numerical error. By restricting the view transform to matmul and element-wise ops, DeployBEV inherits the quantization robustness of these standard operations.

C. Evidence Boundary

Table VII clearly delineates the current evidence boundary. The supported claims are removal of irregular-memory-access operators, extrinsic-free inference, the architectural argument for NPU throughput advantage, and competitive Fisheye3DOD validation accuracy. Claims about final deployment speed numbers, real-world multi-calibration generalization, and cross-dataset transfer are scoped as pending until board-side measurements and real-data experiments are completed. This honest evidence boundary is important to avoid overstating results.

D. Limitations

Several limitations should be acknowledged. First, the current results are primarily on synthetic CARLA data; real-world validation with production sensor data is still in progress. Second, while the architectural argument for NPU throughput advantage (regular blocked memory access vs. data-dependent random access) is sound, board-side latency, memory footprint, and INT8 quantized accuracy numbers on the actual J6 hardware are not yet included. Third, the learned projection matrices are specific to the camera configuration used during training; unlike geometry-based methods, they do not automatically adapt to changed extrinsics without retraining or fine-tuning. Fourth, the mAP is somewhat lower than the geometry-based baseline, particularly for small object classes, suggesting that the purely learned mapping may lose some fine-grained localization ability. Future work will address these through real-data fine-tuning, full deployment benchmarking on J6, and hybrid approaches that combine learned projections with lightweight geometric priors.

VI. Conclusion

This paper presented DeployBEV, a deployable self-learned BEV network for multi-fisheye 3D object detection on embedded NPUs. The central idea is to learn separable height and width projection matrices for the view transformation, replacing `grid_sample`, two-dimensional gather, `scatterND`, `BEVPool`, and any dependence on camera extrinsic parameters at inference time. The key architectural insight is that the deployment bottleneck of BEV view transformation is not merely operator availability but memory access pattern: gather, scatter, and grid sampling perform data-dependent random access that underutilizes NPU MAC arrays and DMA pipelines, whereas matrix multiplication provides regular blocked access that runs at full NPU throughput. On the Fisheye3DOD benchmark, DeployBEV reaches 0.4801 NDS, within 0.0052 of the original geometry-based Fisheye3DOD method, while using 25.8% fewer total parameters and only operators with regular memory access patterns that NPU compilers can map to dense compute. The practical takeaway is that camera-to-BEV transformation should be designed around the memory access characteristics of the target accelerator from the ground up, rather than treated as an after-the-fact model conversion problem. Future work will complete real-data validation, INT8 quantized board-side benchmarking on Horizon J6, systematic ablations on architecture hyperparameters, and the two rank-ceiling extensions outlined in the *PAH* analysis: the affine form $Q = PAH + B$ and the Kronecker-sum form $Q = \sum_k P_k A H_k$.

References

- [1] J. Philion and S. Fidler, “Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d,” in *Proceedings of the European Conference on Computer Vision*, 2020, pp. 195–210. [1](#), [2](#)
- [2] J. Huang, G. Huang, Z. Zhu, Y. Ye, and D. Du, “BEVDet: High-performance multi-camera 3d object detection in bird-eye-view,” in *Proceedings of the European Conference on Computer Vision Workshops*, 2022. [1](#), [2](#)
- [3] C. Li, W. Lin, Z. Hou, G. Chen, W. Zhang, H. Zhou, and W. Zheng, “Exploring surround-view fisheye camera 3d object detection,” *arXiv preprint arXiv:2511.18695*, 2025. [1](#), [3](#), [7](#)
- [4] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as points,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019. [2](#), [3](#), [6](#)
- [5] Y. Liu, T. Wang, X. Zhang, and J. Sun, “PETR: Position embedding transformation for multi-view 3d object detection,” *arXiv preprint arXiv:2203.05621*, 2022. [2](#)
- [6] T. Liang, H. Xie, K. Yu, Z. Xia, Z. Lin, Y. Wang, T.-J. Tang, B. Wang, and Z. Tang, “BEVFusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023. [2](#)
- [7] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, and J. Dai, “BEVFormer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers,” in *Proceedings of the European Conference on Computer Vision*, 2022. [2](#)
- [8] S. Li, X. Chen, Y. Liu, T. Wang, and X. Zhang, “StreamPETR: Exploring object-centric temporal modeling for efficient multi-view 3d object detection,” *arXiv preprint arXiv:2303.11926*, 2023. [2](#)
- [9] H. Zhou, Z. Ge, Z. Li, and X. Zhang, “MatrixVT: Efficient multi-camera to BEV transformation for 3d perception,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023. [2](#)
- [10] H. Zhou, Z. Ge, W. Mao, and Z. Li, “PersDet: Monocular 3d detection in perspective bird’s-eye-view,” in *Proceedings of the British Machine Vision Conference*, 2022. [2](#)
- [11] Y. Li, B. Huang, Z. Chen, Y. Cui, F. Liang, M. Shen, F. Liu, E. Xie, L. Sheng, W. Ouyang, and J. Shao, “Fast-BEV: Towards real-time on-vehicle bird’s-eye view perception,” *arXiv preprint arXiv:2301.02591*, 2023. [3](#)
- [12] T. Pham, M. Maghoumi, W. Jiang, B. S. S. Jujavarapu, M. Sajjadi, X. Liu, H. Lin, B. Chen, G. Truong, C. Fang, J. Kwon, and M. Park, “NVAutoNet: Fast and accurate 360-degree 3d visual perception for self driving,” *arXiv preprint arXiv:2303.12976*, 2023. [3](#)
- [13] C. Yang, T. Lin, L. Huang, and E. J. Crowley, “WidthFormer: Toward efficient transformer-based BEV view transformation,” *arXiv preprint arXiv:2401.03836*, 2024. [3](#)
- [14] J. Kannala and S. S. Brandt, “A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 8, pp. 1335–1340, 2006. [3](#)
- [15] Z. Wu, Y. Gan, X. Li, Y. Wu, X. Wang, T. Xu, and F. Wang, “Surround-view fisheye BEV-perception for valet parking: Dataset, baseline and distortion-insensitive multi-task framework,” *arXiv preprint arXiv:2212.04111*, 2022. [3](#)
- [16] M. Gochoo, M.-E. Otgonbold, E. Ganbold, J.-W. Hsieh, M.-C. Chang, P.-Y. Chen, B. Dorj, H. Al Jassmi, G. Batnasan, F. Alnajjar, M. Abduljabbar, and F.-P. Lin, “FishEye8K: A benchmark and dataset for fisheye camera object detection,” *arXiv preprint arXiv:2305.17449*, 2023. [3](#)
- [17] A. R. Sekkat, Y. Dupuis, V. R. Kumar, H. Rashed, S. Yogamani, P. Vasseur, and P. Honeine, “SynWoodScape: Synthetic surround-view fisheye camera dataset for autonomous driving,” *arXiv preprint arXiv:2203.05056*, 2023. [3](#)
- [18] S. Sonarghare, P. Deshpande, C. Hogan, D.-R. Kaliappan-Mahalingam, and G. Sistu, “FisheyeGaussianLift: BEV feature lifting for surround-view fisheye camera perception,” *arXiv preprint arXiv:2511.17210*, 2025. [3](#)
- [19] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the International Conference on Machine Learning*, 2019, pp. 6105–6114. [3](#), [6](#)
- [20] M. Tan, R. Pang, and Q. V. Le, “EfficientDet: Scalable and efficient object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 10 781–10 790. [3](#), [6](#)
- [21] J. Huang, Y. Ye, Z. Liang, Y. Shan, and D. Du, “Detecting as labeling: Rethinking LiDAR-camera fusion in 3d object detection,” *arXiv preprint arXiv:2311.07152*, 2023. [3](#)
- [22] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nuScenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 11 621–11 631. [7](#), [8](#)

TABLE VII
Evidence status of major claims in this paper.

Claim	Evidence	Status
No irregular-memory-access operators	Code in-spec-tion con-firms only mat-mul, per-mute, soft-max, mul-tiply, add	Supported
No extrinsic parameters needed	No ego2cam/cam2ego/lidar2cam used in forward path	Supported
Regular memory access $\Rightarrow$ NPU throughput	Architectural analysis: mat-mul is blocked/compute-bound; gather/scatter are random/memory-bound	Supported
Competitive Fisheye3DOD NDS	Version 3* achieves 0.4801 vs. 0.4853 base-line	Supported
Better orientation/scale error	mAOE 0.38 vs. 0.48, mASE 0.13 vs. 0.16	Supported
Actual board-side NPU latency	Controlled J6 bench-mark-ing is ongoing	Pending
Real-world multi-extrinsic generalization	Simulation only; real data fine-tuning needed	Pending
nuScenes pinhole generality	Extended vari-ants reach 0.1903 NDS; early-stage train-ing	Preliminary